LECTURE 4

Pandas, Part II

More on `pandas` (Utility Functions, Grouping, Aggregation)

Presented by Dr. Devan Becker, Data200

Adapted from Data 100, Summer 2023 @ UC Berkeley by Bella Crouch and Dominic Liu

Content credit: Lisa Yan, Josh Hug, Suraj Rampure, Allen Shen, Joey Gonzalez, and Sam Lau

CC BY NC SA

Announcements

- Assignment 1 due Jan 28
- Lab 2 on Monday

CC BY NC SA

Goals for this Lecture

Continue our tour of `pandas`

- Extract data according to a condition
- Modify columns in a DataFrame
- Aggregate data

Last lecture: introducing tools

Today: “doing things”

CC BY NC SA

Agenda

- Conditional selection
- Adding, removing, and modifying columns
- Useful utility functions
- Grouping
- Some `.groupby` puzzles

CC BY NC SA

Conditional Selection

- **Conditional selection**
- Adding, removing, and modifying columns
- Useful utility functions
- Grouping
- Some `.groupby` puzzles

CC BY NC SA

Boolean Array Input for `.loc` and `[ ]`

Last time, we extracted data according to its **integer position** (`.iloc`) or its **label** (`.loc`)

What if we want to extract rows that satisfy a given *condition*?

- `.loc` and `[ ]` also accept boolean arrays as input
- Rows corresponding to `True` are extracted; rows corresponding to `False` are not

```
babynames_first_10_rows = babynames.loc[:9, :]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
5	CA	F	1910	Ruth	128
6	CA	F	1910	Evelyn	126
7	CA	F	1910	Alice	118
8	CA	F	1910	Virginia	101
9	CA	F	1910	Elizabeth	93

CC BY NC SA

Boolean Array Input for `.loc` and `[ ]`

- `.loc` and `[ ]` also accept boolean arrays as input
- Rows corresponding to `True` are extracted; rows corresponding to `False` are not

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
5	CA	F	1910	Ruth	128
6	CA	F	1910	Evelyn	126
7	CA	F	1910	Alice	118
8	CA	F	1910	Virginia	101
9	CA	F	1910	Elizabeth	93

```
babynames_first_10_rows[[True, False, True, False,
True, False, True, False, True, False]]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101

CC BY NC SA

Boolean Array Input

We can perform the same operation using `loc`.

```
babynames_first_10_rows.loc[[True, False, True, False, True, False, True,
False, True, False], :]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101

CC BY NC SA

Boolean Array Input

Useful because boolean arrays can be generated by using logical operators on Series.

Length 400761 Series where every entry is either “True” or “False”, where “True” occurs for every babyname with "Sex" = "F".

```
logical_operator = (babynames["Sex"] == "F")
0          True
1          True
2          True
3          True
4          True
          ...
400757    False
400758    False
400759    False
400760    False
400761    False
Name: Sex, Length: 400762, dtype: bool
```

True in rows 0, 1, 2, …

CC BY NC SA

Boolean Array Input

Useful because boolean arrays can be generated by using logical operators on `Series`.

Length 235791 Series where every entry belongs to a babyname with "Sex" = "F"

Length 400761 Series where every entry is either “True” or “False”, where “True” occurs for every baby name with "Sex" = "F".

```
babynames[(babynames["Sex"] == "F")]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
235786	CA	F	2021	Zarahi	5
235787	CA	F	2021	Zelia	5
235788	CA	F	2021	Zenobia	5
235789	CA	F	2021	Zeppelin	5
235790	CA	F	2021	Zoraya	5

235791 rows × 5 columns

CC BY NC SA

Boolean Array Input

Can also use **`.loc`**.

Length 235791 Series where every entry belongs to a babyname with "Sex" = "F"

Length 400761 Series where every entry is either “True” or “False”, where “True” occurs for every babyname with "Sex" = "F".

```
babynames.loc[babynames["Sex"] == "F", :]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
235786	CA	F	2021	Zarahi	5
235787	CA	F	2021	Zelia	5
235788	CA	F	2021	Zenobia	5
235789	CA	F	2021	Zeppelin	5
235790	CA	F	2021	Zoraya	5

235791 rows × 5 columns

CC BY NC SA

Boolean Array Input

Boolean `Series` can be combined using various operators, allowing filtering of results by multiple criteria.

- **The & operator allows us to apply `logical_operator_1` *and* `logical_operator_2`**
- The | operator allows us to apply `logical_operator_1` *or* `logical_operator_2`

```
babynames[(babynames["Sex"] == "F") & (babynames["Year"] < 2000)]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
149044	CA	F	1999	Zareen	5
149045	CA	F	1999	Zeinab	5
149046	CA	F	1999	Zhane	5
149047	CA	F	1999	Zoha	5
149048	CA	F	1999	Zoila	5

149049 rows × 5 columns

Rows that have a Sex of "F" *and* are earlier than the year 2000

CC BY NC SA

Boolean Array Input

Boolean `Series` can be combined using various operators, allowing filtering of results by multiple criteria.

- The **&** operator allows us to apply `logical_operator_1` *and* `logical_operator_2`
- **The | operator allows us to apply `logical_operator_1` *or* `logical_operator_2`**

```
babynames[(babynames["Sex"] == "F") | (babynames["Year"] < 2000)]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
338690	CA	M	1999	Yuuki	5
338691	CA	M	1999	Zakariya	5
338692	CA	M	1999	Zavier	5
338693	CA	M	1999	Zayn	5
338694	CA	M	1999	Zayne	5

338695 rows × 5 columns

Rows that have a Sex of "F" *or* are earlier than the year 2000 (or both!)

CC BY NC SA

Bitwise Operators

`&` and `|` are examples of **bitwise operators**. They allow us to apply multiple logical conditions.

If `p` and `q` are boolean arrays or `Series`:

Symbol	Usage	Meaning
`~`	`~p`	Negation of `p`
`|`	`p | q`	`p` OR `q`
`&`	`p & q`	`p` AND `q`
`^`	`p ^ q`	`p` XOR `q` (exclusive or)

CC BY NC SA

Alternatives to Direct Boolean Array Selection

Boolean array selection is a useful tool, but can lead to overly verbose code for complex conditions.

```
babynames[(babynames["Name"] == "Milad") |
          (babynames["Name"] == "Stephanie") |
          (babynames["Name"] == "Zaid") |
          (babynames["Name"] == "Tina")]
```

	State	Sex	Year	Name	Count
1941	CA	F	1915	Tina	5
2346	CA	F	1916	Tina	6
2891	CA	F	1917	Tina	5
3346	CA	F	1918	Stephanie	6
3401	CA	F	1918	Tina	5
...	...	...	...	...	...
389896	CA	M	2018	Zaid	65
392125	CA	M	2018	Milad	5
392776	CA	M	2019	Zaid	61
395586	CA	M	2020	Zaid	70
398407	CA	M	2021	Zaid	64

269 rows × 5 columns

`pandas` provides **many** alternatives, for example:

- `.isin`
- `.str.startswith`
- `.groupby.filter` (we'll see this in Lecture 5)

CC BY NC SA

Alternatives to Direct Boolean Array Selection

`pandas` provides **many** alternatives, for example:

- `.isin`
- `.str.startswith`
- `.groupby.filter` (see lecture 4)

```
names = ["Milad", "Stephanie", "Zaid", "Tina"]
babynames[babynames["Name"].isin(names)]
```

```
0         False
1         False
2         False
3         False
4         False
          ...
400757    False
400758    False
400759    False
400760    False
400761    False
Name: Name, Length: 400762, dtype: bool
```

Returns a Boolean Series that is True when the corresponding name in `babynames` is Milad, Stephanie, Zaid, or Tina

CC BY NC SA

Alternatives to Boolean Array Selection

`pandas` provides **many** alternatives, for example:

- `.isin`
- **`.str.startswith`**
- `.groupby.filter` (see lecture 4)

```
babynames[babynames["Name"].str.startswith("N")]
```

Returns a Boolean Series that is True when the corresponding name in **`babynames`** starts with "N"

```
0         False
1         False
2         False
3         False
4         False
          ...
400757    False
400758    False
400759    False
400760    False
400761    False
Name: Name, Length: 400762, dtype: bool
```

	State	Sex	Year	Name	Count
76	CA	F	1910	Norma	23
83	CA	F	1910	Nellie	20
127	CA	F	1910	Nina	11
198	CA	F	1910	Nora	6
310	CA	F	1911	Nellie	23
...	...	...	...	...	...
400648	CA	M	2021	Nirvan	5
400649	CA	M	2021	Nivin	5
400650	CA	M	2021	Nolen	5
400651	CA	M	2021	Nomar	5
400652	CA	M	2021	Nyles	5

11994 rows × 5 columns

CC BY NC SA

Adding, Removing, and Modifying Columns

- Conditional selection
- **Adding, removing, and modifying columns**
- Useful utility functions
- Grouping
- Some `.groupby` puzzles

CC BY NC SA

Syntax for Adding a Column

Adding a column is easy:

1. Use `[ ]` to reference the desired new column
2. Assign this column to a `Series` or array of the appropriate length

```
# Create a Series of the length of each name; we'll discuss `str` next week
babyname_lengths = babynames["Name"].str.len()

# Add a column named "name_lengths" that includes the length of each name
babynames["name_lengths"] = babyname_lengths
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7

CC BY NC SA

Syntax for Modifying a Column

Modifying a column is very similar to adding a column.

1. Use `[ ]` to reference the existing column
2. Assign this column to a new Series or array of the appropriate length

```
# Modify the “name_lengths” column to be one less than its original value
babynames["name_lengths"] = babynames["name_lengths"]-1
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6

CC BY NC SA

Syntax for Renaming a Column

Rename a column using the (creatively named) `.rename()` method

- `.rename()` takes in a **dictionary** that maps old column names to their new ones

```
# Rename “name_lengths” to “Length”
babynames = babynames.rename(columns={“name_lengths”:“Length”})
```

	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6

CC BY NC SA

Syntax for Dropping a Column (or Row)

Remove columns using the (also creatively named) `.drop` method

- The `.drop()` method assumes you're dropping a row by default. Use `axis = "columns"` to drop a column instead.

```
babynames = babynames.drop("Length", axis = "columns")
```

	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

CC BY NC SA

An Important Note: DataFrame Copies

Notice that we *reassigned* `babynames` to an updated value on the previous slide.

```
babynames = babynames.drop("Length", axis = "columns")
```

By default, `pandas` methods create a **copy** of the DataFrame, without changing the original DataFrame at all. To apply our changes, we must update our DataFrame to this new, modified copy.

```
babynames.drop("Length", axis = "columns")
babynames
```

	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6

Our change was not applied!

CC BY NC SA

Useful Utility Functions

- Conditional selection
- Adding, removing, and modifying columns
- **Useful utility functions**
- Grouping
- Some `.groupby` puzzles

CC BY NC SA

NumPy

Pandas **Series** and **DataFrames** support a large number of operations, including mathematical operations, so long as the data is numerical. [Data 8 NumPy reference.]

```
yash_count = babynames[babynames["Name"] == "Yash"]["Count"]
```

```
np.mean(yash_count)
```

```
17.296296296296298
```

```
np.max(yash_count)
```

```
29
```

```
328079     8
330369     9
332645    11
335028    12
337642    10
339826    14
342022    24
344485    29
347144    24
349700    29
352476    25
355233    27
358086    29
361160    24
364122    23
367200    18
370309    14
373009    18
375911    18
379588     9
382154    12
384780    17
387736    16
391156    10
394121     9
396416    15
399329    13
Name: Count, dtype: int64
```


CC BY NC SA

Built-In `pandas` Methods

In addition to its rich syntax for indexing and support for other libraries (`NumPy`, native Python functions), `pandas` provides an enormous number of useful utility functions. Today, we'll discuss just a few:

- `size/shape`
- `describe`
- `sample`
- `value_counts`
- `unique`
- `sort_values`

The `pandas` library is rich in utility functions (we could spend the entire summer talking about them)! We encourage you to explore as you complete your assignments by Googling and reading documentation, just as data scientists do.

CC BY NC SA

`.shape` and `.size`

- `.shape` returns the shape of a DataFrame or Series in the form (number of rows, number of columns
- `.size` returns the total number of entries in a DataFrame or Series (number of rows x number of columns)

```
babynames
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
400757	CA	M	2021	Zyan	5
400758	CA	M	2021	Zyion	5
400759	CA	M	2021	Zyire	5
400760	CA	M	2021	Zylo	5
400761	CA	M	2021	Zyrus	5

400762 rows × 5 columns

```
babynames.shape
(400762, 5)
```

```
babynames.size
2003810
```


CC BY NC SA

.describe()

- `.describe()` returns a “description” of a DataFrame or Series that lists summary statistics of the data

```
babynames
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
400757	CA	M	2021	Zyan	5
400758	CA	M	2021	Zyion	5
400759	CA	M	2021	Zyire	5
400760	CA	M	2021	Zylo	5
400761	CA	M	2021	Zyrus	5

400762 rows × 5 columns

```
babynames.describe()
```

	Year	Count
count	400762.000000	400762.000000
mean	1985.131287	79.953781
std	26.821004	295.414618
min	1910.000000	5.000000
25%	1968.000000	7.000000
50%	1991.000000	13.000000
75%	2007.000000	38.000000
max	2021.000000	8262.000000

CC BY NC SA

.describe()

- A different set of statistics will be reported if **`.describe()`** is called on a **`Series`**.

```
babynames["Sex"].describe()
```

```
count     400762
unique         2
top            F
freq      235791
Name: Sex, dtype: object
```


CC BY NC SA

.sample()

To sample a random selection of rows from a `DataFrame`, we use the `.sample()` method.

- By default, ***it is without replacement***. Use `replace=True` for **replacement**.
- Naturally, can be chained with other methods and operators (`iloc`, etc).

```
babynames.sample()
```

	State	Sex	Year	Name	Count
108418	CA	F	1988	Janielle	6

```
babynames.sample(5).iloc[:, 2:]
```

	Year	Name	Count
169346	2005	Greta	36
231690	2020	Yui	6
203404	2013	Libby	14
385359	2016	Cael	9
386609	2017	Everett	353

```
babynames[babynames["Year"] == 2000]
          .sample(4, replace=True)
          .iloc[:, 2:]
```

	Year	Name	Count
340297	2000	Emmet	8
339662	2000	Fred	17
150463	2000	Rosalia	18
152732	2000	Shanae	5

CC BY NC SA

.value_counts()

The **`Series.value_counts`** method counts the number of occurrences of each unique value in a **`Series`** (it *counts* the number of times each *value* appears).

- Return value is also a **`Series`**.

```
babyname["Name"].value_counts()
```

```
Jean         221
Francis      219
Guadalupe    216
Jessie       215
Marion       213
            ...
Janin          1
Jilliann       1
Jomayra        1
Karess         1
Zyrus          1
Name: Name, Length: 20239, dtype: int64
```


CC BY NC SA

.unique()

The **`Series.unique`** method returns an array of every unique value in a **`Series`**.

```
babynames["Name"].unique()
```

```
array(['Mary', 'Helen', 'Dorothy', ..., 'Zyire', 'Zylo', 'Zyrus'],
      dtype=object)
```


CC BY NC SA

.sort_values()

The `DataFrame.sort_values` and `Series.sort_values` methods sort a `DataFrame` (or `Series`).

- **`Series.sort_values( )` will automatically sort all values in the Series**
- `DataFrame.sort_values(column_name)` must specify the name of the column to be used for sorting

```
babynames["Name"].sort_values()
```

```
380256      Aadan
362255      Aadan
365374      Aadan
394460    Aadarsh
366561      Aaden
           ...
232144      Zyrah
217415      Zyrah
197519      Zyrah
220674      Zyrah
400761      Zyrus
Name: Name, Length: 400762, dtype: object
```


CC BY NC SA

.sort_values()

The `DataFrame.sort_values` and `Series.sort_values` methods sort a `DataFrame` (or `Series`).

- `Series.sort_values( )` will automatically sort all values in the Series
- **`DataFrame.sort_values(column_name)` must specify the name of the column to be used for sorting**

```
babynames.sort_values(by = "Count", ascending=False)
```

	State	Sex	Year	Name	Count
263272	CA	M	1956	Michael	8262
264297	CA	M	1957	Michael	8250
313644	CA	M	1990	Michael	8247
278109	CA	M	1969	Michael	8244
279405	CA	M	1970	Michael	8197
...	...	...	...	...	...
159967	CA	F	2002	Arista	5
159966	CA	F	2002	Arisbeth	5
159965	CA	F	2002	Arisa	5
159964	CA	F	2002	Arionna	5
400761	CA	M	2021	Zyrus	5

400762 rows × 5 columns

By default, rows are sorted in *ascending* order

CC BY NC SA

Grouping

- Conditional selection
- Adding, removing, and modifying columns
- Useful utility functions
- **Grouping**
- Some `.groupby` puzzles

CC BY NC SA

Why Group?

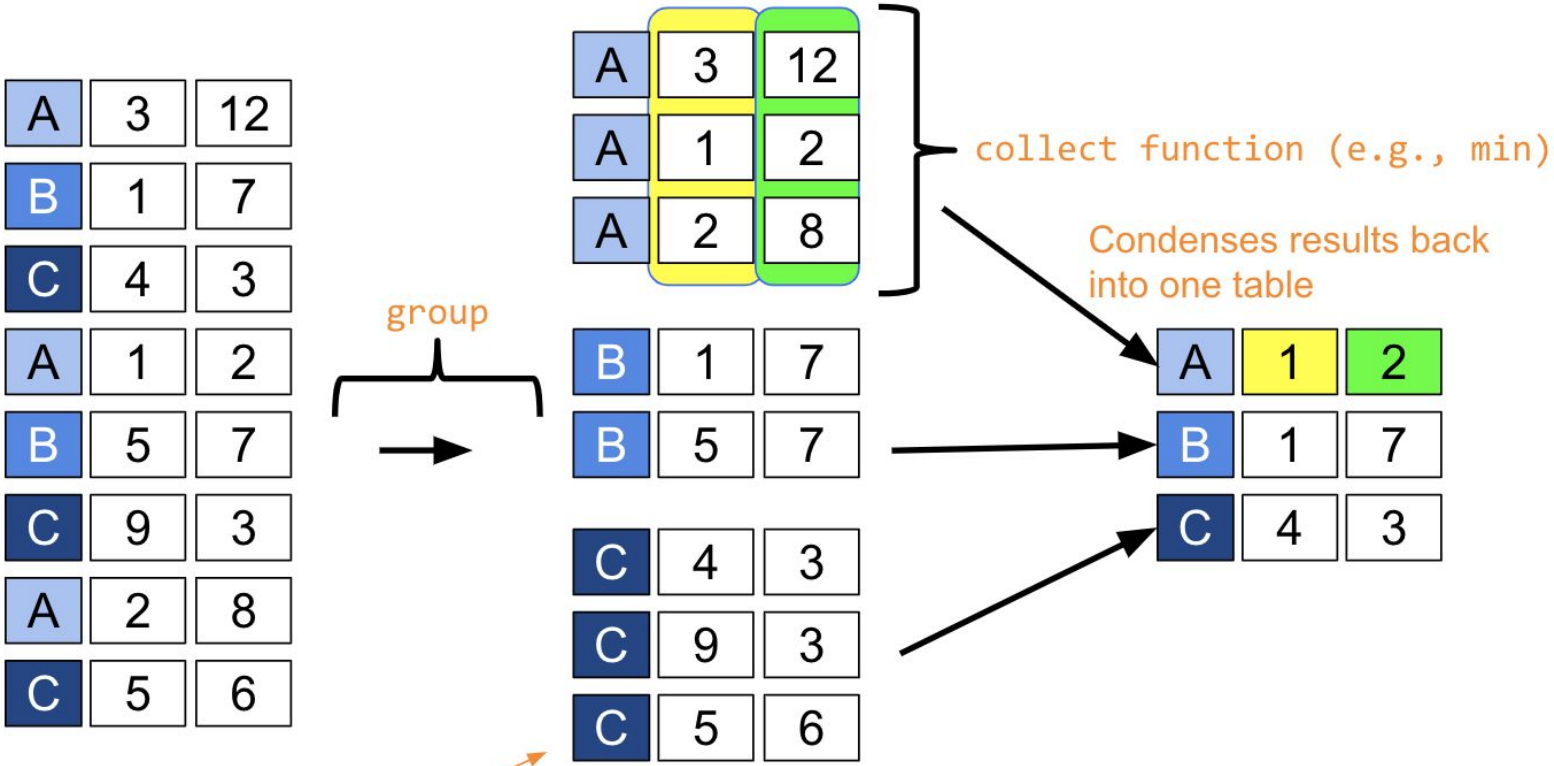
Our goal:

- Group together rows that fall under the same category
  - For example, group together all rows from the same year
- Perform an operation that *aggregates* across all rows in the category
  - For example, sum up the total number of babies born in that year

Grouping is a powerful tool to 1) perform large operations, all at once and 2) summarize trends in a dataset

CC BY NC SA

Grouping and Collection



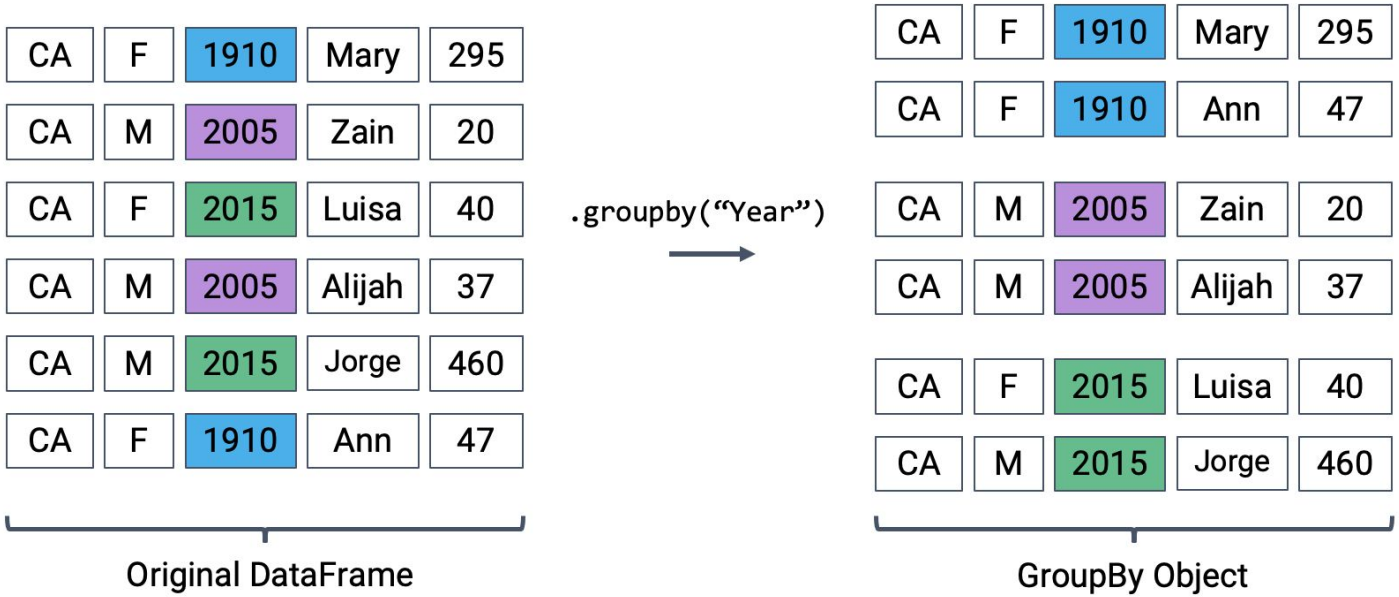
Can think of as temporary (A,B,C) sub-tables

CC BY NC SA

.groupby()

A .groupby() operation involves some combination of **splitting the object, applying a function**, and **combining the results**.

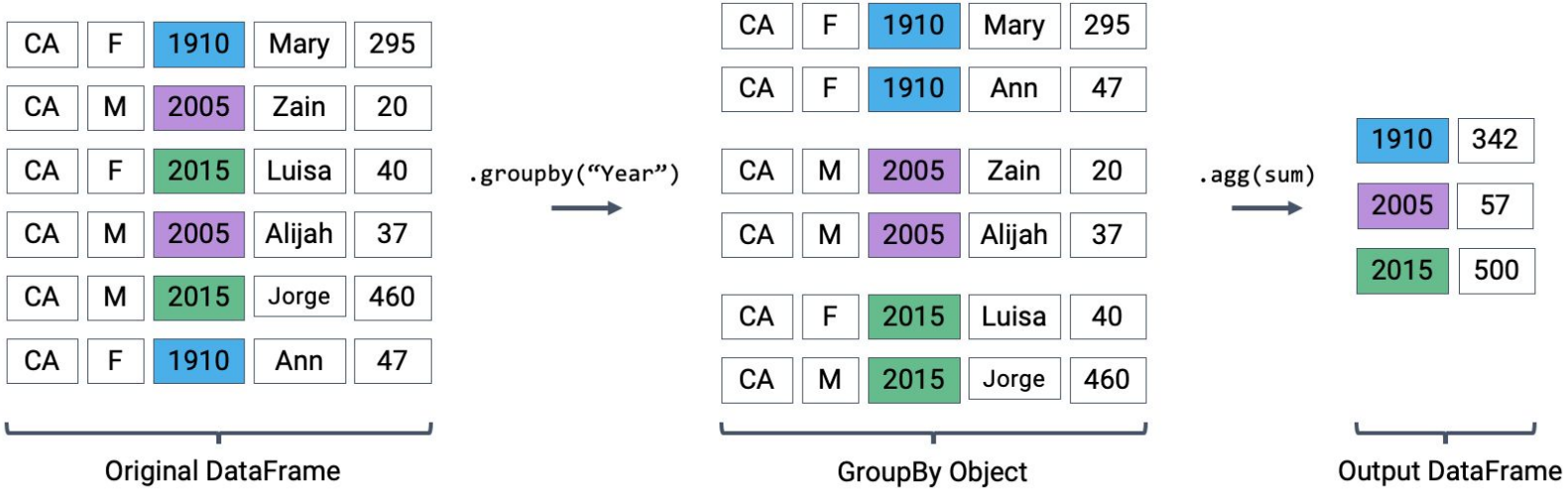
- Calling .groupby() generates DataFrameGroupBy objects → "mini" sub-DataFrames
- Each subframe contains all rows that correspond to the same group (here, a particular year)



CC BY NC SA

`.groupby().agg()`

- We cannot work directly with **`DataFrameGroupBy`** objects! The diagram below is to help understand what goes on conceptually – in reality, we can't "see" the result of calling **`.groupby`**
- Instead, we transform a **`DataFrameGroupBy`** object back into a DataFrame using **`.agg`**
  - **`.agg`** is how we apply an aggregation operation to the data



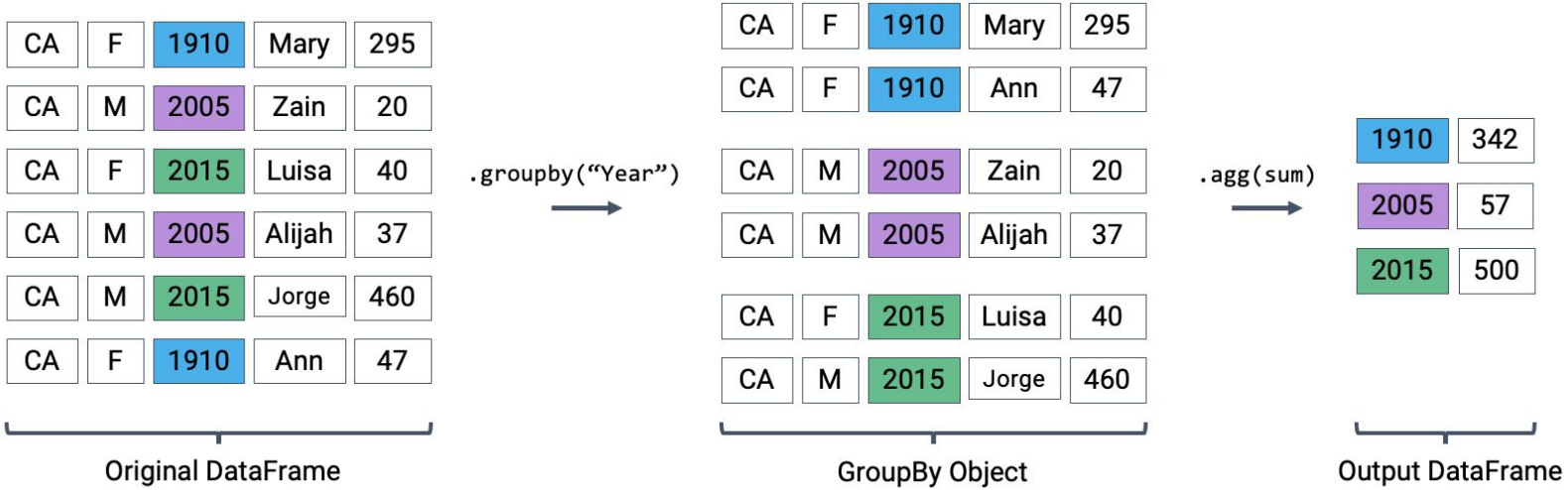
Where did the non-numeric columns go? We'll talk about this in a bit.

CC BY NC SA

Putting It All Together

```
dataframe.groupby(column_name).agg(aggregation_function)
```

- `babynames.groupby("Year").agg(sum)` computes the total number of babies born in each year.



CC BY NC SA

Aggregation Functions

What goes inside of `.agg( )`?

- Any function that aggregates several values into one summary value
- Common examples:

In-Built Python Functions	NumPy Functions	In-Built `pandas` functions
`.agg(sum)`	`.agg(np.sum)`	`.agg("sum")`
`.agg(max)`	`.agg(np.max)`	`.agg("max")`
`.agg(min)`	`.agg(np.min)`	`.agg("min")`
	`.agg(np.mean)`	`.agg("mean")`
		`.agg("first")`
		`.agg("last")`

Some commonly-used aggregation functions can even be called directly, without the explicit use of `.agg( )`

```
babynames.groupby("Year").mean()
```

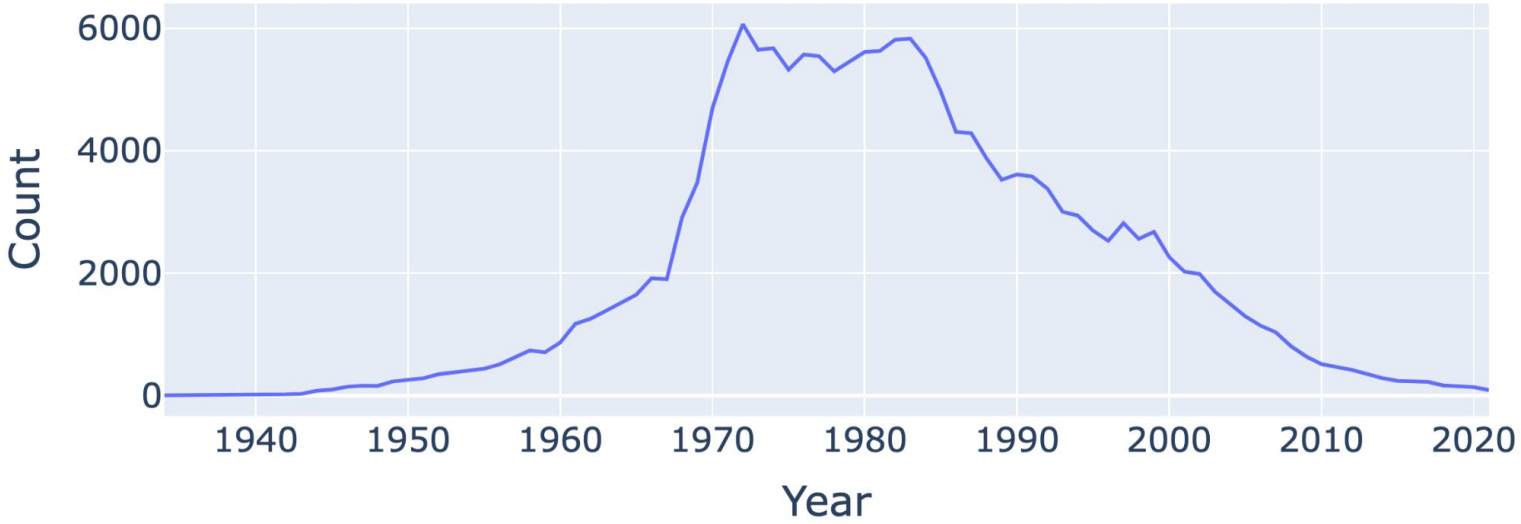

CC BY NC SA

Putting Things Into Practice

Goal: Find the baby name with sex “F” that has fallen in popularity the most

```
f_babynames = babynames[babynames["Sex"] == "F"]
f_babynames = f_babynames.sort_values("Year")
jenn_counts_series = f_babynames[f_babynames["Name"] == "Jennifer"]["Count"]
```

Number of Jennifers born in California each year



CC BY NC SA

What Is “Popularity”?

Goal: Find the baby name with sex “F” that has fallen in popularity the most

How do we define “fallen in popularity?”

- Let’s create a metric: “ratio to peak” (RTP)
- The RTP is the ratio of babies born with a given name in 2021 to the *maximum* number of babies born with that name in *any* year

Example for "Jennifer":

- In 1972, we hit peak Jennifer. 6,065 Jennifers were born.
- In 2021, there were only 91 Jennifers.
- RTP is 91 / 6065 = 0.015004.

CC BY NC SA

Calculating RTP

```
max_jenn = max(f_babynames[f_babynames["Name"] == "Jennifer"]["Count"])
```

```
6065
```

```
curr_jenn = f_babynames[f_babynames["Name"] == "Jennifer"]["Count"].iloc[-1]
```

```
91
```

Remember: `f_babynames` is sorted by year.
`.iloc[-1]` means "grab the latest year"

```
rtp = curr_jenn / max_jenn
```

```
0.015004122011541632
```

```
def ratio_to_peak(series):
    return series.iloc[-1] / max(series)
```

```
jenn_counts_ser = f_babynames[f_babynames["Name"] == "Jennifer"]["Count"]
ratio_to_peak(jenn_counts_ser)
```

```
0.015004122011541632
```


CC BY NC SA

Calculating RTP Using `.groupby()`

`.groupby()` makes it easy to compute the RTP for all names at once!

```
rtp_table = female_babynames.groupby("Name").agg(ratio_to_peak)
```

Name	Year	Count
Aadhira	1.0	0.700000
Aadhya	1.0	0.580000
Aadya	1.0	0.724138
Aahana	1.0	0.192308
Aahna	1.0	1.000000
...	...	...
Zyanya	1.0	0.857143
Zyla	1.0	1.000000
Zylah	1.0	1.000000
Zyra	1.0	1.000000

CC BY NC SA

Answer

In the five rows shown, note the Year is 1.0 for every value.

Are there any rows for which Year is **not** 1.0?

A. Yes, names that appeared for the first time in 2021.
B. Yes, names that did not appear in 2021.
C. Yes, names whose peak Count was in 2021.
D. No, every row has a Year value of 1.0.

```
rtp_table = (
                female_babynames
                .groupby("Name")
                .agg(ratio_to_peak)
                )
```

Name	Year	Count
Aadhira	1.0	0.700000
Aadhya	1.0	0.580000
Aadya	1.0	0.724138
Aahana	1.0	0.192308
Aahna	1.0	1.000000
...	...	...
Zyanya	1.0	0.857143
Zyla	1.0	1.000000
Zylah	1.0	1.000000
Zyra	1.0	1.000000

CC BY NC SA

A Note on Nuisance Columns

At least as of the time of this slide creation (June 2023), executing our agg call results in:

```
female_babynames.groupby("Name").agg(ratio_to_peak)
```

```
/tmp/ipykernel_116/1062259356.py:4: FutureWarning:

['State', 'Sex'] did not aggregate successfully. If any error is raised this will raise in a future version of pandas.
Drop these columns/ops to avoid this warning.
```

	State	Sex	Year	Name	Count
1	CA	F	1910	Helen	239
83	CA	F	1910	Nellie	20
184	CA	F	1910	Aileen	6
186	CA	F	1910	Astrid	6
187	CA	F	1910	Beulah	6
...	...	...	...	...	...
235340	CA	F	2021	An	5
232380	CA	F	2021	Adelyn	144
232496	CA	F	2021	Ailani	99
232501	CA	F	2021	Dayana	98
235759	CA	F	2021	Vy	5

235791 rows × 5 columns

	Year	Count
Name		
Aadan	1.0	0.714286
Aadarsh	1.0	1.000000
Aaden	1.0	0.063291
Aadhav	1.0	0.750000
Aadhira	1.0	0.700000
...	...	...
Zymir	1.0	1.000000
Zyon	1.0	0.933333
Zyra	1.0	1.000000
Zyrah	1.0	0.833333
Zyrus	1.0	1.000000

20239 rows × 2 columns

We can't "do math" on non-numeric columns, so **pandas** will ignore them.

CC BY NC SA

A Note on Nuisance Columns

Below, we explicitly select the column(s) we want to apply our aggregation function to **BEFORE** calling **agg**. This avoids the warning (and can prevent unintentional loss of data).

```
rtp_table = female_babynames.groupby("Name")[["Count"]].agg(ratio_to_peak)
```

Name	Count
Aadhira	0.700000
Aadhya	0.580000
Aadya	0.724138
Aahana	0.192308
Aahna	1.000000
...	...
Zyanya	0.857143
Zyla	1.000000
Zylah	1.000000
Zyra	1.000000
Zyrah	0.833333

13661 rows × 1 columns

CC BY NC SA

Renaming Columns After Grouping

By default, .groupby will not rename any aggregated columns (the column is still named "Count", even though it now represents the RTP.

For better readability, we may wish to rename "Count" to "Count RTP"

```
rtp_table = female_babynames.groupby("Name")[["Count"]].agg(ratio_to_peak)
rtp_table = rtp_table.rename(columns = {"Count": "Count RTP"})
```

Name	Count
Aadhira	0.700000
Aadhya	0.580000
Aadya	0.724138
Aahana	0.192308
Aahna	1.000000

Name	Count RTP
Aadhira	0.700000
Aadhya	0.580000
Aadya	0.724138
Aahana	0.192308
Aahna	1.000000

CC BY NC SA

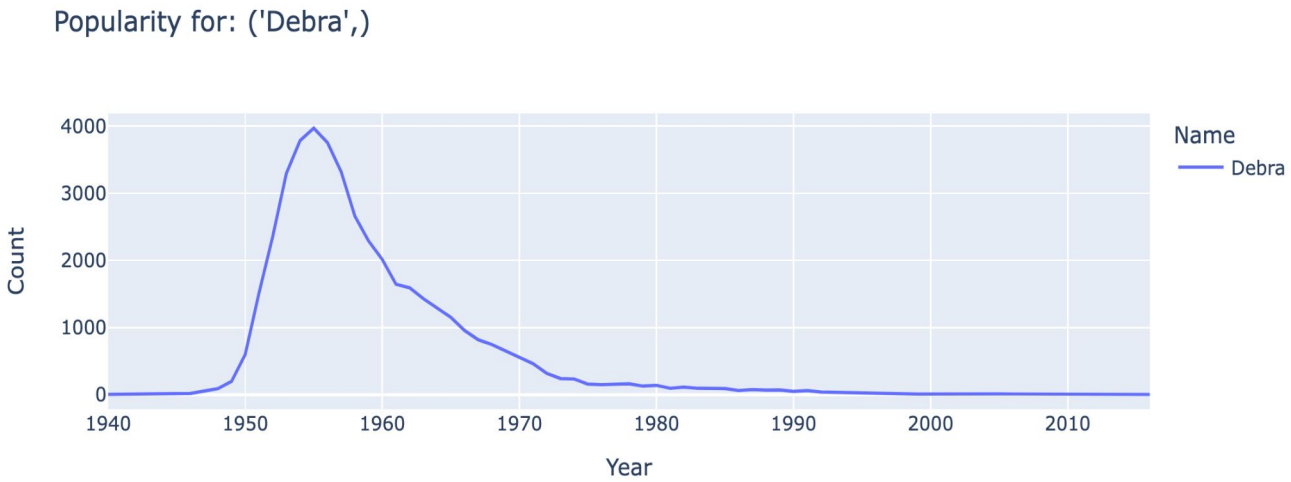
Some Data Science Payoff

By sorting `rtp_table` we can see the names that decreased in popularity the most.

```
rtp_table.sort_values("Count RTP")
```

Name	Count RTP
Debra	0.001260
Susan	0.002034
Debbie	0.002817
Cheryl	0.003273
Carol	0.003635
...	...
Jovi	1.000000
Neta	1.000000
Doni	1.000000
Dondi	1.000000
Kela	1.000000

13661 rows × 1 columns



We'll learn about plotting in Lecture 7.

CC BY NC SA

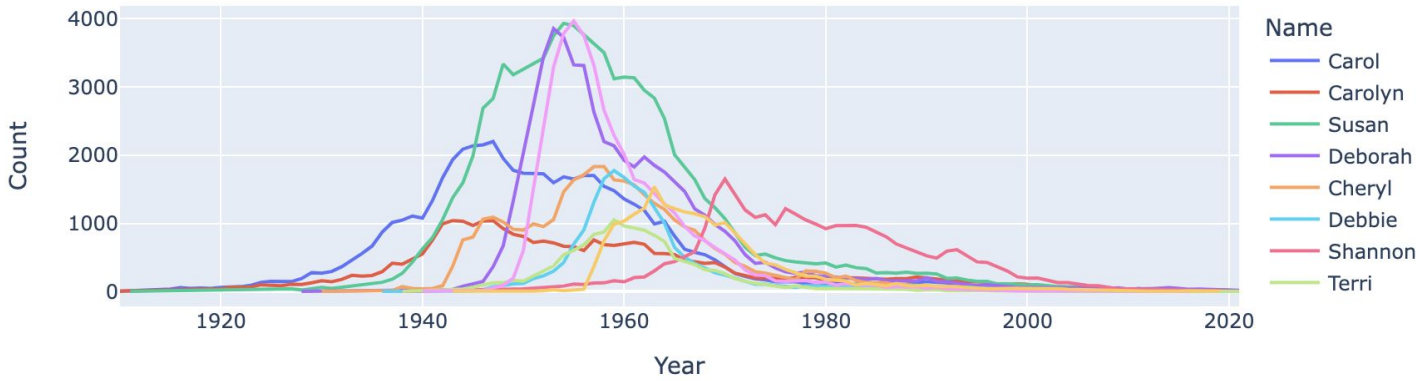
Some Data Science Payoff

We can get the list of the top 10 names and then plot popularity with::

```
top10 = rtp_table.sort_values("Count RTP").head(10).index
```

```
Index(['Debra', 'Susan', 'Debbie', 'Cheryl', 'Carol', 'Tammy', 'Terri',
       'Shannon', 'Deborah', 'Carolyn'],
      dtype='object', name='Name')
```

```
px.line(female_babynames[female_babyname["Name"].isin(top10)],
                                x = "Year", y = "Count", color = "Name")
```



CC BY NC SA

Some `.groupby` puzzles

- Conditional selection
- Adding, removing, and modifying columns
- Useful utility functions
- Grouping
- **Some `.groupby` puzzles**

CC BY NC SA

Q1

Which line of code will compute the total number of babies with each name?

```
A.  f_babynames.groupby("Count")[["Name"]].sum()
B.  f_babynames.groupby("Count")[["Name"]].agg(sum)
C.  f_babynames.groupby("Name")[["Count"]].sum()
D.  f_babynames.groupby("Name")[["Count"]].agg(sum())
```


CC BY NC SA

Q1 Notes

Before we saw that the code below generates the Count RTP for all female names.

```
f_babynames.groupby("Name")[["Count"]].agg(ratio_to_peak)
```

We use similar logic to compute the summed counts of all female names.

```
f_babynames.groupby("Name")[["Count"]].agg(sum)
```

or

```
f_babynames.groupby(“Name”)[[“Count”]].sum()
```


CC BY NC SA

Q2

Which line of code will compute the total number of babies born each *year*?

```
A.  female_babynames.groupby("Year")[["Count"]].agg(sum)
B.  female_babynames.groupby("Year")[["Count"]].sum
C.  female_babynames.groupby("Name")[["Year"]].agg(sum)
```


CC BY NC SA

Q2 Notes

Now, we create groups for each *year*.

```
female_babynames.groupby("Year")[["Count"]].agg(sum)
```

or

```
female_babynames.groupby("Name")[["Count"]].sum()
```

Year	Count
1910	5950
1911	6602
1912	9804
1913	11860
1914	13815
...	...
2017	195466
2018	189066
2019	183982
2020	173428
2021	172693

112 rows × 1 columns

CC BY NC SA

Q3

Which of the following best describes the groupby-aggregate approach?

A. Find summaries of a category.
B. Find summaries of one or more variables according to groupings of another.
C. Find summaries of all variables according to groupings of one variable.
D. Calculate statistics based on our knowledge of the problem.

CC BY NC SA

Q4

Which of the following is *not* a requirement of the function inside of `.agg()`?

A. Takes a DataFrame or Series as an input.
B. Performs an operation on one or more of the columns.
C. Produces an output of length one.

CC BY NC SA

Q5

Which of the following is correct syntax?

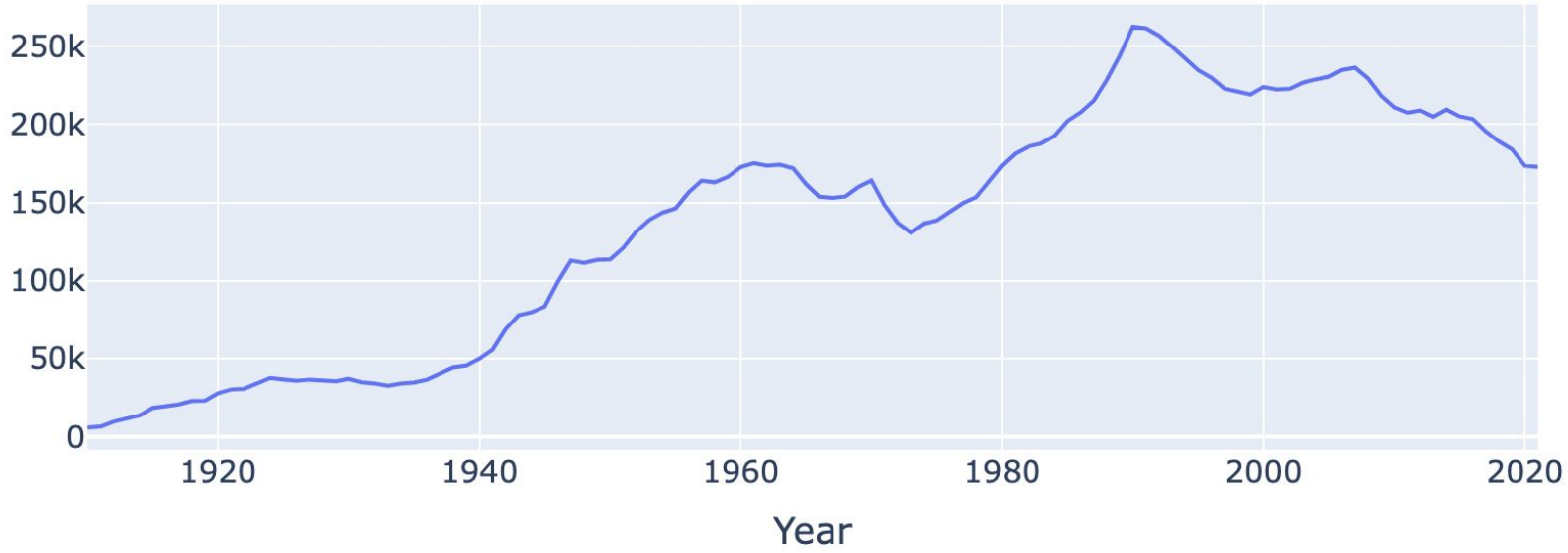
```
A.  elections[elections["Party"] == "Republican" or "Democrat"]
B.  elections[elections["Party"] == "Republican" or elections["Party"] == "Democrat"]
C.  elections[elections["Party"] == "Republican" | "Democrat"]
D.  elections[elections["Party"] == "Republican" | elections["Party"] == "Democrat"]
```


CC BY NC SA

Plotting Birth Counts

Plotting the **`DataFrame`** we just generated tells an interesting story.

```
puzzle2 = female_babynames.groupby("Year")[["Count"]].agg(sum)
px.line(puzzle2, y = "Count")
```

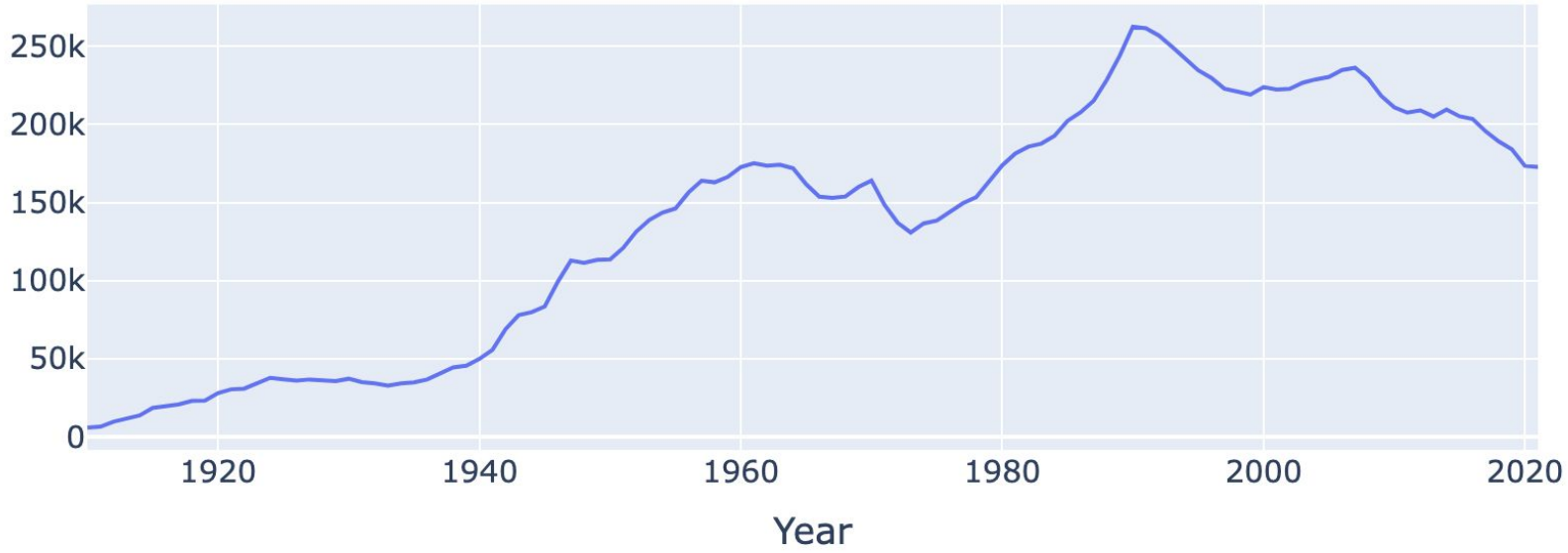


CC BY NC SA

A Word of Warning!

We made an enormous assumption when we decided to use this dataset to estimate the birth rate.

- According to https://lao.ca.gov/LAOEconTax/Article/Detail/691, the true number of babies born in California in 2020 was 421,275.
- What happened?

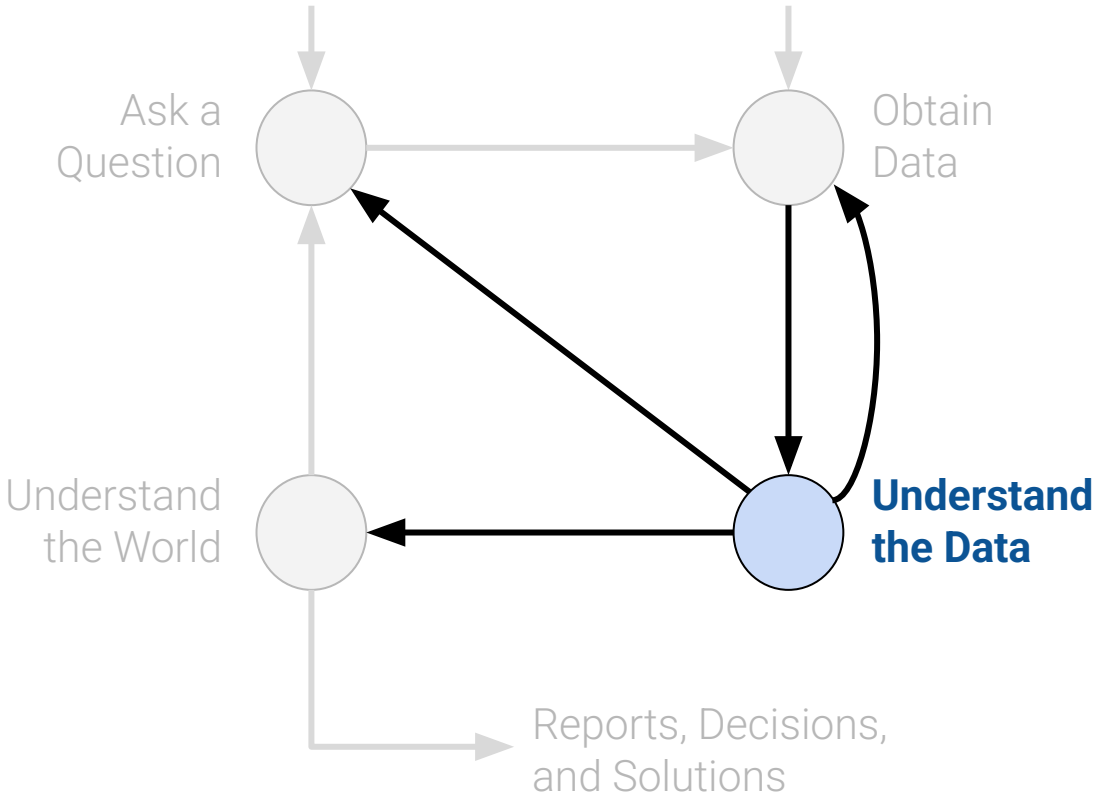


CC BY NC SA

Understanding the Data

- How is our data organized and what does it contain?
- Do we already have relevant data?
- What are the biases, anomalies, or other issues with the data?
- How do we transform the data to enable effective analysis?

Bottom line: Blindly using tools is dangerous!

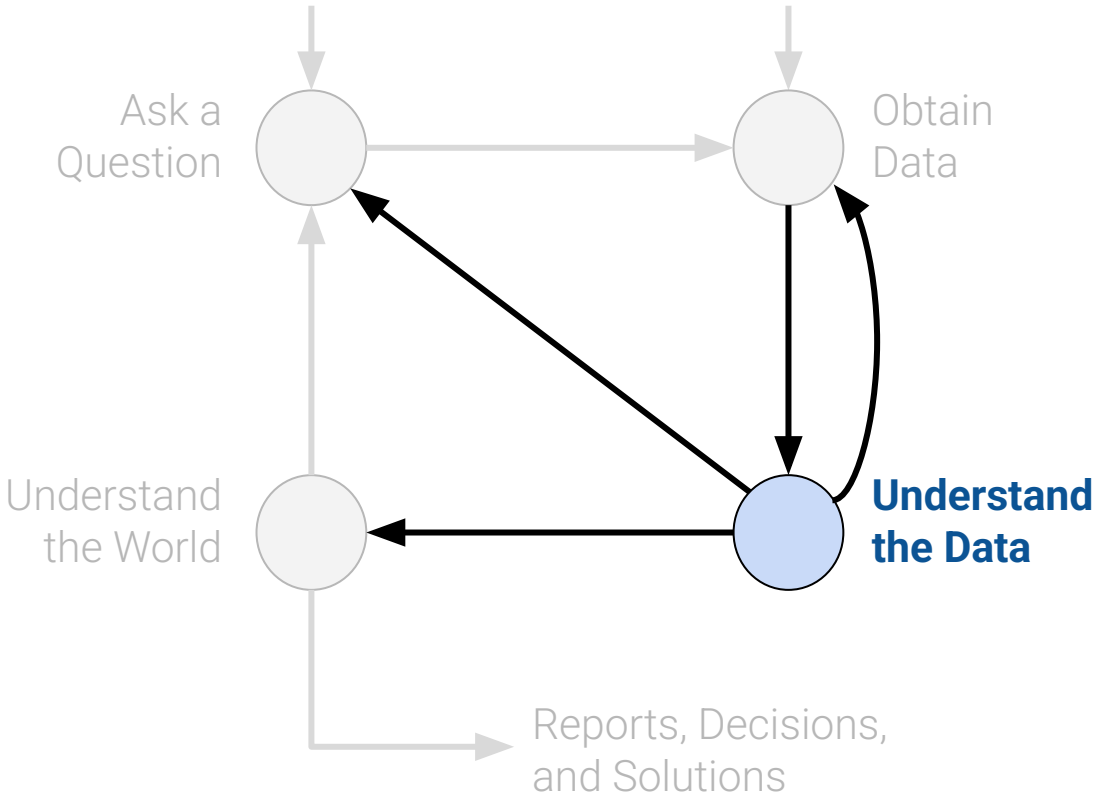


CC BY NC SA

Understanding the Data

What are the biases, anomalies, or other issues with the data?

- Not all babies register for social security.
- The database does not include names with fewer than 5 entries per year (seeing this in action: many of our TAs' names are not represented in the data!)



CC BY NC SA

A fun little data science personal project?

FUTURE

What is BBC Future? Future Planet Inner Space Follow the Food More

IN DEPTH | PSYCHOLOGY

Why millennials are choosing strange baby names

(Image credit: Alamy)

EVERYDAY Food Money Travel Family Work

Why parents are giving their kids 'unique' baby names

ABC Everyday / By Grace Jennings-Edquist

Popular Latest *The Atlantic* Sign In

FAMILY

The Age of the Unique Baby Name

Parents used to want kids to fit in. Now they want them to stand out.

By Joe Pinsker

- Are there enough unique baby names in recent years to skew these data significantly?
- How would you test that?

CC BY NC SA